

# FastAPI Backend

Complete Developer Guide  
From Zero to Production-Ready API

---

|                       |                                                  |
|-----------------------|--------------------------------------------------|
| Written for           | C++ / Systems Engineers learning Python Backend  |
| Stack covered         | Python, FastAPI, Pydantic, SQLAlchemy, SQLite    |
| What you will build   | Full CRUD Task Manager API with real database    |
| Next steps after this | PostgreSQL, Deployment, JWT Auth, AI Integration |

*Everything in this guide was built step by step. Every command, every function, every reason. Read it once to understand. Read it again to teach.*

---

## Section 0 — The Big Picture

Before writing a single line of code, understand what you are building and why each tool exists. This section is the mental map — every section after this will make more sense if you read this first.

### What is a Backend?

A backend is a program that sits on a server, waits for requests from the outside world, does some work (read/write data, run logic, call other services), and sends back a response. Every app you use — WhatsApp, Uber, Spotify — has a backend doing exactly this.

Without backend: User → types in your terminal → Python CLI runs

With backend: User → sends HTTP request from anywhere → your Python API runs

Same Python logic. Different entry point. Different scale.

### The Tools and Why Each One

| Tool       | What it is            | Why we use it                                                         |
|------------|-----------------------|-----------------------------------------------------------------------|
| Python     | Programming language  | AI/ML ecosystem lives here. FastAPI is Python.                        |
| FastAPI    | Web framework         | Turns Python functions into HTTP endpoints. Fastest Python framework. |
| Pydantic   | Data validation       | Validates incoming data automatically. Built into FastAPI.            |
| SQLAlchemy | Database ORM          | Lets you talk to databases using Python instead of raw SQL.           |
| SQLite     | Database (learning)   | Zero setup. Single file. Perfect for learning SQL concepts.           |
| PostgreSQL | Database (production) | What real companies use. Same SQLAlchemy code works.                  |
| Uvicorn    | ASGI Server           | Runs your FastAPI app and listens for incoming requests.              |
| Alembic    | DB migrations         | Tracks database changes over time. Like Git for your database.        |

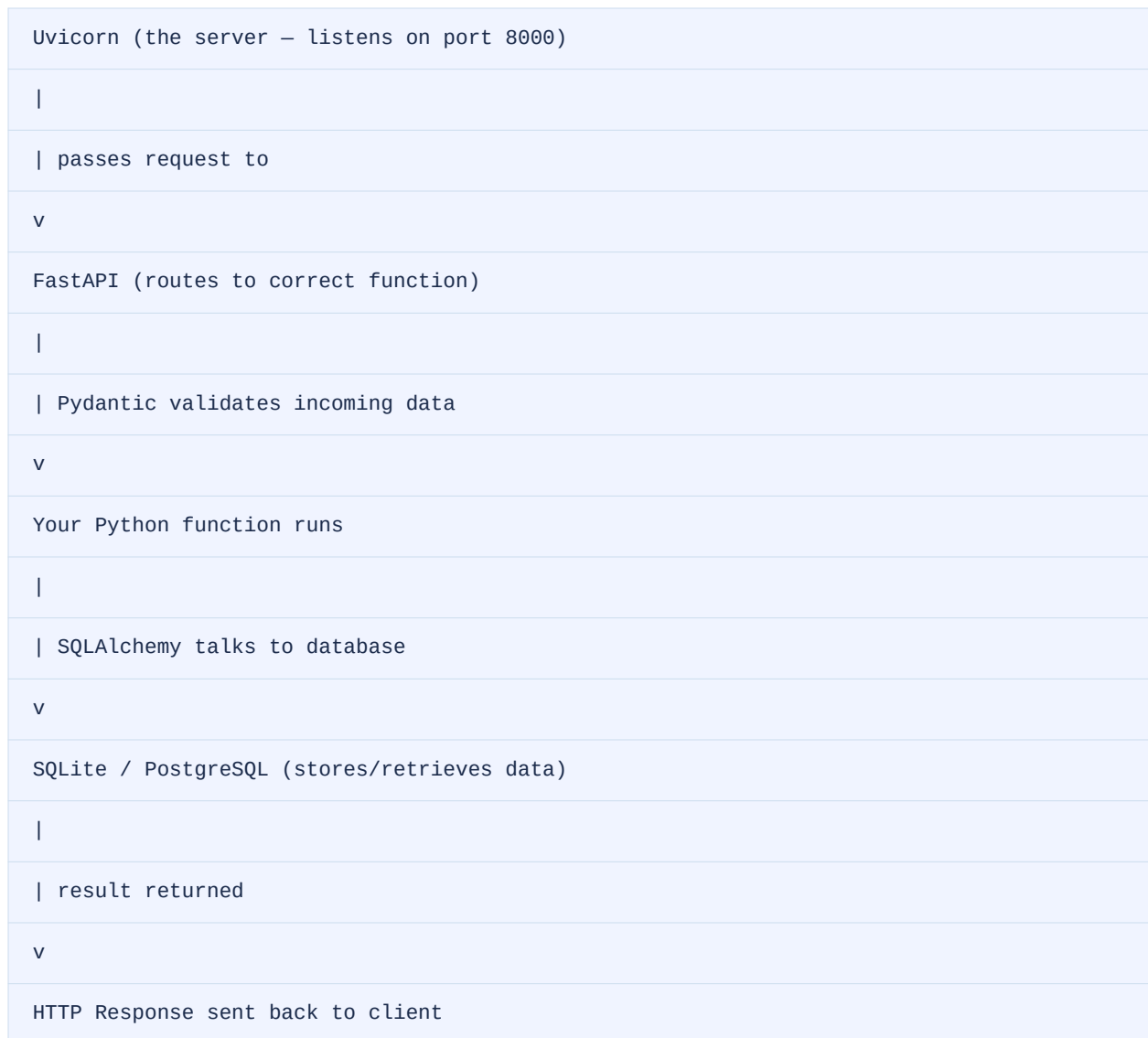
### The Request Flow — How Everything Connects

Client (browser / mobile app / another server)

|

| HTTP Request (GET /tasks)

v



---

## Section 1 — Project Setup

### Folder Structure

Create this folder structure on your computer. Everything lives in one folder.

|                                              |
|----------------------------------------------|
| task_manager/                                |
| main.py ← CLI version (built first)          |
| api.py ← FastAPI version (built second)      |
| database.py ← SQLAlchemy setup (built third) |
| data.json ← auto-created by CLI version      |
| tasks.db ← auto-created by SQLAlchemy        |

### Install Dependencies

Run these commands in your terminal. Make sure you are inside your project folder first.

|                                              |
|----------------------------------------------|
| # Navigate to your project folder            |
| cd "C:\Users\YourName\Desktop\task_manager"  |
|                                              |
| # Install FastAPI and its server             |
| pip install fastapi uvicorn                  |
|                                              |
| # Install SQLAlchemy for database            |
| pip install sqlalchemy                       |
|                                              |
| # Install Alembic for migrations (use later) |
| pip install alembic                          |
|                                              |
| # Verify installations                       |
| pip show fastapi sqlalchemy uvicorn          |

**Note:** Always run `pip install` in your project terminal, not VS Code debugger. Use `Ctrl+`` to open terminal in VS Code.

## How to Run Your API

```
# Standard command

uvicorn api:app --reload

# If uvicorn not recognized

python -m uvicorn api:app --reload

# What each part means:

# api = the file name (api.py)

# app = the FastAPI() object inside api.py

# --reload = auto-restart when you save changes

# Your API runs at:

# http://localhost:8000 main URL

# http://localhost:8000/docs interactive UI (Swagger)
```

---

## Section 2 — CLI Project (main.py)

We built the CLI version first. This taught us Python fundamentals — functions, dictionaries, file I/O, error handling — before adding the web layer. Never skip this step. The CLI logic becomes your API logic.

### Why CLI First?

| CLI version teaches you       | API version uses it as               |
|-------------------------------|--------------------------------------|
| load_tasks() — read from JSON | Same logic, now reads from database  |
| save_tasks() — write to JSON  | Same logic, now writes to database   |
| Task dictionary structure     | Same structure, now a database table |
| Input validation with if/else | Pydantic does this automatically     |
| Menu loop with while True     | Replaced by HTTP endpoints           |

### Complete main.py — CLI Version



[illegible]

[illegible]

```

def main():

    tasks = load_tasks()

    while True:

        show_menu()

        choice = input('Choose (1-5): ').strip()

        if choice == '1': add_task(tasks)

        elif choice == '2': tasks = load_tasks(); view_tasks(tasks)

        elif choice == '3': mark_task_done()

        elif choice == '4': delete_task()

        elif choice == '5': print('Goodbye!'); break

        else: print('Invalid choice.')

    if __name__ == '__main__': # run only when executed directly

        main()

```

## Key Python Concepts Used

| Concept                   | Where used              | What it does                                   |
|---------------------------|-------------------------|------------------------------------------------|
| json.load() / json.dump() | load_tasks / save_tasks | Read/write Python dict to JSON file            |
| os.path.exists()          | load_tasks              | Check if file exists before reading            |
| input().strip()           | All user input          | Get input and remove whitespace                |
| List comprehension        | delete_task             | Filter list in one line                        |
| if __name__ == '__main__' | Bottom of file          | Only run main() when file is executed directly |
| f-strings                 | Everywhere              | Embed variables in strings cleanly             |

---

## Section 3 — HTTP & REST API Concepts

Before writing FastAPI code, understand the foundation it is built on. These concepts apply to every backend framework in every language.

### What is HTTP?

HTTP is the language of the web. Every request from a browser, mobile app, or API call uses HTTP. It is just a standardized way to send and receive data over the internet.

```
# A raw HTTP request looks like this:
```

```
GET /tasks HTTP/1.1
```

```
Host: localhost:8000
```

```
# A raw HTTP response looks like this:
```

```
HTTP/1.1 200 OK
```

```
Content-Type: application/json
```

```
{"total": 3, "tasks": [...]}
```

### The 4 HTTP Methods — CRUD

| HTTP Method | CRUD Operation | Our Endpoint       | What it does                  |
|-------------|----------------|--------------------|-------------------------------|
| GET         | Read           | GET /tasks         | Fetch all tasks from database |
| POST        | Create         | POST /tasks        | Add a new task to database    |
| PUT         | Update         | PUT /tasks/{id}    | Mark a task done/pending      |
| DELETE      | Delete         | DELETE /tasks/{id} | Remove a task from database   |

### HTTP Status Codes

| Code                     | Meaning              | When we use it                     |
|--------------------------|----------------------|------------------------------------|
| 200 OK                   | Success              | GET, PUT requests succeed          |
| 201 Created              | Created successfully | POST request creates a task        |
| 404 Not Found            | Resource missing     | Task ID does not exist in database |
| 422 Unprocessable Entity | Validation failed    | Pydantic rejects bad request data  |

|                  |                   |                                  |
|------------------|-------------------|----------------------------------|
| 500 Server Error | Something crashed | Unhandled exception in your code |
|------------------|-------------------|----------------------------------|

## What is REST?

REST is a set of rules for designing APIs. You do not need to memorize all rules, but the core idea is: use URLs to identify resources and HTTP methods to define actions.

|                                               |
|-----------------------------------------------|
| # REST approach – clean and predictable       |
| GET /tasks → get all tasks                    |
| POST /tasks → create a task                   |
| GET /tasks/5 → get task with id 5             |
| PUT /tasks/5 → update task with id 5          |
| DELETE /tasks/5 → delete task with id 5       |
| # Non-REST approach – messy and unpredictable |
| GET /getAllTasks                              |
| POST /createNewTask                           |
| GET /getTaskById?id=5                         |
| POST /updateTaskStatus                        |
| POST /deleteTask                              |

---

## Section 4 — FastAPI (api.py — JSON version first)

We built the API in two phases. First with JSON file storage (same as CLI), then upgraded to SQLAlchemy database. This section covers the first phase — understanding FastAPI concepts before adding database complexity.

### What FastAPI Does

FastAPI takes your Python functions and exposes them to the internet as HTTP endpoints. You write normal Python functions. FastAPI handles the web stuff.

#### FastAPI vs C++

| C++                      | Python                          |
|--------------------------|---------------------------------|
| // No equivalent in C++  | from fastapi import FastAPI     |
| // You would need a full | app = FastAPI()                 |
| // web server framework  |                                 |
| // like Crow or Pistache | @app.get('/tasks')              |
|                          | def get_tasks():                |
|                          | return load_tasks()             |
|                          | # That's it. Function is now    |
|                          | # accessible at /tasks via HTTP |

### Core FastAPI Concepts

| Concept       | What it is                        | Example                             |
|---------------|-----------------------------------|-------------------------------------|
| FastAPI()     | Creates your app instance         | app = FastAPI()                     |
| @app.get()    | Decorator — links URL to function | @app.get('/tasks')                  |
| @app.post()   | Handles POST requests             | @app.post('/tasks')                 |
| @app.put()    | Handles PUT requests              | @app.put('/tasks/{id}')             |
| @app.delete() | Handles DELETE requests           | @app.delete('/tasks/{id}')          |
| {task_id}     | Path parameter in URL             | /tasks/{task_id} → task_id variable |
| return dict   | FastAPI auto-converts to JSON     | return {'message': 'ok'}            |

### What is a Decorator?

A decorator is the @ symbol above a function. It wraps your function with extra behaviour — in FastAPI's case, it registers your function as an HTTP endpoint.

```
@app.get('/tasks') # decorator – registers this function for GET /tasks

def get_tasks(): # your normal Python function

    return {} # FastAPI converts this dict to JSON automatically


# Without decorator – just a normal function

def get_tasks(): # only callable from Python

    return {}


# With decorator – accessible from the internet

@app.get('/tasks') # now GET http://localhost:8000/tasks calls this

def get_tasks():

    return {}
```

## Pydantic — Request Validation

Pydantic validates incoming data automatically. You define what data you expect using a class. FastAPI uses it to reject bad requests before they reach your function.

```
from pydantic import BaseModel


# Define what a valid Task request looks like

class TaskCreate(BaseModel):

    title: str # must exist, must be string

    deadline: str # must exist, must be string


# Use it in endpoint – FastAPI validates automatically

@app.post('/tasks')

def add_task(task: TaskCreate): # task is validated before this runs

    print(task.title) # guaranteed to exist and be a string

    print(task.deadline) # guaranteed to exist and be a string
```

```
# If someone sends {"tile": "Learn"} → 422 error automatically

# If someone sends {"title": 12345} → 422 error automatically

# If someone sends {} → 422 error automatically

# Your function only runs if data is perfect
```

**Note:** *Pydantic replaces all your manual if/else validation checks from the CLI version. One class = all validation handled.*

## HTTPException — Returning Errors

```
from fastapi import HTTPException

# Return a proper HTTP error with status code and message

raise HTTPException(status_code=404, detail='Task not found')

# This sends back:

# HTTP 404 {"detail": "Task not found"}

# Common status codes:

# 404 = not found

# 400 = bad request

# 401 = unauthorized

# 403 = forbidden

# 500 = server error
```

## Complete api.py — JSON Version

```
from fastapi import FastAPI, HTTPException

from pydantic import BaseModel

import json, os

app = FastAPI()

DATA_FILE = 'data.json'
```

[illegible]

```
'title': task.title, # from validated Pydantic model
```

```
'deadline': task.deadline,
```

```
'done': False
```

```
tasks.append(new_task)
```

```
save_tasks(tasks)
```

```
return {'message': 'Task added', 'task': new_task}
```

[illegible]

```
@app.put('/tasks/{task_id}') # {task_id} = URL parameter
```

```
def update_task(task_id: int, task_data: TaskUpdate):
```

```
tasks = load_tasks()
```

```
for record in tasks:
```

```
if record.get('id') == task_id:
```

```
record['done'] = task_data.done
```

```
save_tasks(tasks)
```

```
return {'message': f'Task {task_id} updated', 'task': record}
```

```
raise HTTPException(status_code=404, detail='Task not found')
```

[illegible]

```
@app.delete('/tasks/{task_id}')
```

```
def delete_task(task_id: int):
```

```
tasks = load_tasks()
```

```
original_count = len(tasks)
```

```
tasks = [r for r in tasks if r.get('id') != task_id]
```

```
if len(tasks) < original_count:
```

```
save_tasks(tasks)
```

```
return {'message': f'Task {task_id} deleted'}
```

```
raise HTTPException(status_code=404, detail='Task not found')
```

---

## Section 5 — Database with SQLAlchemy

We replaced the JSON file with a real database. The API logic stays the same — only the storage layer changes. This is the power of SQLAlchemy.

### Why Replace JSON with a Database?

| Problem with JSON file                                 | Database solution                             |
|--------------------------------------------------------|-----------------------------------------------|
| No concurrent access — two requests writing at once    | Database handles concurrent access safely     |
| No querying — you load everything and filter in Python | Query only what you need: filter, sort, limit |
| No relationships — linking tasks to users is manual    | Foreign keys and joins built in               |
| Grows slow — loading 1 million tasks to find one       | Indexed lookups — instant regardless of size  |
| Not production ready                                   | SQLite for dev, PostgreSQL for production     |

### SQLAlchemy vs SQLite vs PostgreSQL — Cleared Up

This confuses everyone. Here is the clear distinction:

| Thing      | Category       | What it does                                                                |
|------------|----------------|-----------------------------------------------------------------------------|
| SQLite     | DATABASE       | Stores your data in a single .db file. Zero setup. Built into Python.       |
| PostgreSQL | DATABASE       | Stores your data on a database server. Used in production.                  |
| SQLAlchemy | ORM TOOL       | Python library to talk to databases. Works with both SQLite and PostgreSQL. |
| Alembic    | MIGRATION TOOL | Tracks and applies database schema changes over time.                       |

```
# The relationship:
```

```
Your Python code
```

```
|
```

```
| uses
```

```
v
```

```
SQLAlchemy (ORM) ← you write Python, it writes SQL
```

```
|
```

```
| connects to
```

```
v
```

|                                                                       |
|-----------------------------------------------------------------------|
| SQLite (learning) OR PostgreSQL (production)                          |
| # Switching from SQLite to PostgreSQL = ONE LINE CHANGE:              |
| DATABASE_URL = 'sqlite:///tasks.db' # SQLite                          |
| DATABASE_URL = 'postgresql://user:pass@localhost/taskdb' # PostgreSQL |
| # Everything else stays IDENTICAL                                     |

```
# Switching from SQLite to PostgreSQL = ONE LINE CHANGE:
```

```
DATABASE_URL = 'sqlite:///tasks.db' # SQLite
```

```
DATABASE_URL = 'postgresql://user:pass@localhost/taskdb' # PostgreSQL
```

```
# Everything else stays IDENTICAL
```

## database.py — Complete File with Explanation

```
from sqlalchemy import create_engine, Column, Integer, String, Boolean
```

```
from sqlalchemy.ext.declarative import declarative_base
```

```
from sqlalchemy.orm import sessionmaker
```

```
# ■■ STEP 1: DATABASE URL ████████████████████████████████████████
```

```
# Tells SQLAlchemy WHERE the database is
```

```
# sqlite:/// = local file
```

```
# tasks.db = the filename (created automatically)
```

```
DATABASE_URL = 'sqlite:///tasks.db'
```

```
# ■■ STEP 2: ENGINE ████████████████████████████████████████████████████████
```

```
# The actual connection to your database
```

```
# Like a phone line between Python and SQLite
```

```
# check_same_thread=False - allows FastAPI multiple concurrent requests
```

```
engine = create_engine(DATABASE_URL, connect_args={'check_same_thread': False})
```

```
# ■■ STEP 3: SESSIONLOCAL ████████████████████████████████████████████
```

```
# A factory that creates database sessions
```

```
# Each API request gets its own temporary session
```

```
# Think: each customer at a bank gets their own teller
```

```
SessionLocal = sessionmaker(bind=engine)
```

[illegible][illegible][illegible][illegible][illegible][illegible][illegible]

```
# ■■■ STEP 4: BASE ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
```

```
# Parent class for all your database models
```

```
# Keeps track of all tables you define
```

```
Base = declarative_base()
```

```
# ■■■ STEP 5: TABLE DEFINITION ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
```

```
# This class = one table in your database
```

```
# Each Column = one column in the table
```

```
class TaskModel(Base):
```

```
    __tablename__ = 'tasks' # table name in database
```

```
    id = Column(Integer, primary_key=True, index=True) # auto-increment
```

```
    title = Column(String, nullable=False) # cannot be empty
```

```
    deadline = Column(String, nullable=False) # cannot be empty
```

```
    done = Column(Boolean, default=False) # default not done
```

```
# ■■■ STEP 6: CREATE TABLES ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
```

```
# Reads all classes that inherit from Base
```

```
# Creates actual tables in database if they don't exist
```

```
# Safe to call multiple times – won't recreate existing tables
```

```
def init_db():
```

```
    Base.metadata.create_all(bind=engine)
```

[illegible]

```
# ■■ STEP 4: BASE ████████████████████████████████████████████████████

# Parent class for all your database models

# Keeps track of all tables you define

Base = declarative_base()


■■ STEP 5: TABLE DEFINITION ████████████████████████████████████████

# This class = one table in your database

# Each Column = one column in the table

class TaskModel(Base):

    __tablename__ = 'tasks' # table name in database

    id = Column(Integer, primary_key=True, index=True) # auto-increment

    title = Column(String, nullable=False) # cannot be empty

    deadline = Column(String, nullable=False) # cannot be empty

    done = Column(Boolean, default=False) # default not done



■■ STEP 6: CREATE TABLES ████████████████████████████████████████████

# Reads all classes that inherit from Base

# Creates actual tables in database if they don't exist

# Safe to call multiple times – won't recreate existing tables

def init_db():

    Base.metadata.create_all(bind=engine)
```

```
# ■■■ STEP 4: BASE ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
```

```
# Parent class for all your database models
```

```
# Keeps track of all tables you define
```

```
Base = declarative_base()
```

```
# ■■■ STEP 5: TABLE DEFINITION ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
```

```
# This class = one table in your database
```

```
# Each Column = one column in the table
```

```
class TaskModel(Base):
```

```
    __tablename__ = 'tasks' # table name in database
```

```
    id = Column(Integer, primary_key=True, index=True) # auto-increment
```

```
    title = Column(String, nullable=False) # cannot be empty
```

```
    deadline = Column(String, nullable=False) # cannot be empty
```

```
    done = Column(Boolean, default=False) # default not done
```

```
# ■■■ STEP 6: CREATE TABLES ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
```

```
# Reads all classes that inherit from Base
```

```
# Creates actual tables in database if they don't exist
```

```
# Safe to call multiple times – won't recreate existing tables
```

```
def init_db():
```

```
    Base.metadata.create_all(bind=engine)
```

[illegible][illegible][illegible][illegible][illegible][illegible]

```
# ■■■ STEP 4: BASE ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
```

```
# Parent class for all your database models
```

```
# Keeps track of all tables you define
```

```
Base = declarative_base()
```

```
# ■■■ STEP 5: TABLE DEFINITION ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
```

```
# This class = one table in your database
```

```
# Each Column = one column in the table
```

```
class TaskModel(Base):
```

```
    __tablename__ = 'tasks' # table name in database
```

```
    id = Column(Integer, primary_key=True, index=True) # auto-increment
```

```
    title = Column(String, nullable=False) # cannot be empty
```

```
    deadline = Column(String, nullable=False) # cannot be empty
```

```
    done = Column(Boolean, default=False) # default not done
```

```
# ■■■ STEP 6: CREATE TABLES ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
```

```
# Reads all classes that inherit from Base
```

```
# Creates actual tables in database if they don't exist
```

```
# Safe to call multiple times – won't recreate existing tables
```

```
def init_db():
```

```
    Base.metadata.create_all(bind=engine)
```

[illegible]

## What Happens When You Run the Server

|                                                                     |
|---------------------------------------------------------------------|
| <pre>uvicorn api:app --reload</pre>                                 |
| Step 1: Python imports database.py                                  |
| Step 2: engine = create_engine() → connection to SQLite established |
| Step 3: SessionLocal = sessionmaker() → session factory ready       |

|                                                                     |
|---------------------------------------------------------------------|
| <pre>uvicorn api:app --reload</pre>                                 |
| Step 1: Python imports database.py                                  |
| Step 2: engine = create_engine() → connection to SQLite established |
| Step 3: SessionLocal = sessionmaker() → session factory ready       |

|                                                                     |
|---------------------------------------------------------------------|
| <pre>uvicorn api:app --reload</pre>                                 |
| Step 1: Python imports database.py                                  |
| Step 2: engine = create_engine() → connection to SQLite established |
| Step 3: SessionLocal = sessionmaker() → session factory ready       |

|                                                                     |
|---------------------------------------------------------------------|
| <pre>uvicorn api:app --reload</pre>                                 |
| Step 1: Python imports database.py                                  |
| Step 2: engine = create_engine() → connection to SQLite established |
| Step 3: SessionLocal = sessionmaker() → session factory ready       |

Step 4: Base = declarative\_base() → table registry created

Step 5: TaskModel class defined → table blueprint registered

Step 6: init\_db() called → tasks.db file created → tasks table created

Step 7: Uvicorn starts listening on port 8000

Step 8: Ready to receive requests

Result: tasks.db file appears in your project folder

## get\_db() — The Session Dependency

Every database operation needs a session. get\_db() creates one per request and ensures it always closes — even if the request crashes.

```
def get_db():  
  
    db = SessionLocal() # Step 1: open a new session  
  
    try:  
  
        yield db # Step 2: give session to the endpoint function  
  
        # yield = 'here, use this, I'll wait'  
  
    finally:  
  
        db.close() # Step 3: ALWAYS close – even if error occurred  
  
  
    # yield vs return:  
  
    # return → gives value and exits forever  
  
    # yield → gives value, pauses, resumes after caller is done  
  
    # This ensures cleanup (db.close) always runs
```

## Depends() — Dependency Injection

Depends() tells FastAPI: before running this endpoint function, run get\_db() first and pass the result as the db parameter. This is called dependency injection.

```
from fastapi import Depends  
  
from sqlalchemy.orm import Session
```

```
@app.get('/tasks')

def get_tasks(db: Session = Depends(get_db)):

    # ^^^^^^^^^^^^^^^^^^^^

    # FastAPI calls get_db() automatically

    # passes the session as 'db'

    # closes it after function returns

    tasks = db.query(TaskModel).all()

    return tasks

# Every endpoint that needs database access gets this parameter

# FastAPI handles the lifecycle – you just use 'db'
```

## The 4 Database Operations

[illegible]

[illegible]

## Why commit() and refresh()?

| Method          | What it does                                      | When to use                             |
|-----------------|---------------------------------------------------|-----------------------------------------|
| db.add(obj)     | Stages new record in memory — not saved yet       | Before every INSERT                     |
| db.commit()     | Writes all staged changes to database permanently | After add, update, or delete            |
| db.refresh(obj) | Reloads object from database into Python memory   | After commit — to get auto-generated id |
| db.close()      | Closes the session and releases connection        | Always — done by get_db() finally block |

**Note:** Without `commit()`, your changes exist only in memory and disappear when the session closes. `commit()` = save permanently.

## Complete api.py — SQLAlchemy Version

[illegible]

```
class TaskCreate(BaseModel):
```

```
title: str
```

```
deadline: str
```

```
class TaskUpdate(BaseModel):
```

```
done: bool
```

[illegible]

```
def get_db():
```

```
db = SessionLocal()
```

```
yield db
```

```
finally:
```

```
db.close()
```

[illegible]

```
@app.get('/tasks')
```

```
def get_tasks(db: Session = Depends(get_db)):
```

```
tasks = db.query(TaskModel).all()
```

```
return {'total': len(tasks), 'tasks': tasks}
```

[illegible]

```
@app.post('/tasks')
```

```
def add_task(task: TaskCreate, db: Session = Depends(get_db)):
```

```
new_task = TaskModel(
```

```
title=task.title,
```

```
deadline=task.deadline,
```

```
done=False
```

```
db.add(new_task)
```

```
db.commit()
```

```
db.refresh(new_task) # get auto-generated id
```

```
return {'message': 'Task added', 'task': new_task}
```

[illegible]

```
@app.put('/tasks/{task_id}')
```

```
def update_task(task_id: int, task_data: TaskUpdate, db: Session = Depends(get_db)):
```

```
task = db.query(TaskModel).filter(TaskModel.id == task_id).first()
```

```
if not task:
```

```
raise HTTPException(status_code=404, detail='Task not found')
```

```
task.done = task_data.done
```

```
db.commit()
```

```
db.refresh(task)
```

```
return {'message': f'Task {task_id} updated', 'task': task}
```

[illegible]

```
@app.delete('/tasks/{task_id}')
```

```
def delete_task(task_id: int, db: Session = Depends(get_db)):
```

```
task = db.query(TaskModel).filter(TaskModel.id == task_id).first()
```

```
if not task:
```

```
raise HTTPException(status_code=404, detail='Task not found')
```

```
db.delete(task)
```

```
db.commit()
```

```
return {'message': f'Task {task_id} deleted'}
```

---

## Section 6 — Testing Your API

---

### Swagger UI — /docs

FastAPI automatically generates an interactive API documentation page. This is your primary testing tool during development.

```
# Run your server

uvicorn api:app --reload


# Open in browser

http://localhost:8000/docs


# What you will see:

# Every endpoint listed with method and URL

# Click any endpoint → Try it out → fill data → Execute

# See real request and response


# Also available:

http://localhost:8000/redoc # alternative docs UI
```

### Testing Each Endpoint

| Endpoint        | How to test in /docs                                                              | Expected response                                      |
|-----------------|-----------------------------------------------------------------------------------|--------------------------------------------------------|
| GET /tasks      | Click → Execute (no input needed)                                                 | {"total": 0, "tasks": []}                              |
| POST /tasks     | Click → Try it out → enter {"title": "Learn", "deadline": "2020-08-01"} → Execute | 201 Created {"message": "Task created", "task": {...}} |
| PUT /tasks/1    | Click → Try it out → task_id=1 → enter {"done": true} → Execute                   | 200 OK {"message": "Task 1 updated", ...}              |
| DELETE /tasks/1 | Click → Try it out → task_id=1 → Execute                                          | 200 OK {"message": "Task 1 deleted"}                   |

### Testing Error Cases

```
# Test 404 – update task that doesn't exist

PUT /tasks/9999 → should return 404 Task not found
```

# Test 422 – send invalid data

POST /tasks with {"tile": "wrong key"} → should return 422

# Test empty – get tasks when database is empty

GET /tasks → should return {"total": 0, "tasks": []}

---

## Section 7 — Key Concepts Quick Reference

---

### Python Concepts Used in This Project

| Concept                         | Syntax                                     | Used where                                   |
|---------------------------------|--------------------------------------------|----------------------------------------------|
| f-strings                       | <code>f"Hello {name}"</code>               | All print/return statements                  |
| List comprehension              | <code>[x for x in lst if condition]</code> | <code>delete_task</code> filtering           |
| Dictionary                      | <code>{"key": "value"}</code>              | Task structure in JSON version               |
| <code>with open()</code>        | <code>with open('f.txt','r') as f:</code>  | File read/write                              |
| <code>try/except/finally</code> | <code>try: ... except: ... finally:</code> | <code>get_db()</code> session cleanup        |
| <code>*args/**kwargs</code>     | <code>def fn(*args, **kwargs):</code>      | Not used here but know it                    |
| <code>yield</code>              | <code>yield value</code>                   | <code>get_db()</code> — gives session, waits |
| Decorator (@)                   | <code>@app.get('/path')</code>             | Every FastAPI endpoint                       |
| Type hints                      | <code>def fn(x: int, y: str):</code>       | Pydantic models, function params             |

### SQLAlchemy Cheat Sheet

| Operation     | SQLAlchemy code                                             | SQL equivalent                                      |
|---------------|-------------------------------------------------------------|-----------------------------------------------------|
| Get all       | <code>db.query(Model).all()</code>                          | <code>SELECT * FROM table</code>                    |
| Get one by id | <code>db.query(Model).filter(Model.id==id).first()</code>   | <code>SELECT * FROM table WHERE id=? LIMIT 1</code> |
| Filter        | <code>db.query(Model).filter(Model.done==True).all()</code> | <code>SELECT * FROM table WHERE done=True</code>    |
| Add new       | <code>db.add(obj); db.commit()</code>                       | <code>INSERT INTO table VALUES (...)</code>         |
| Update        | <code>obj.field = value; db.commit()</code>                 | <code>UPDATE table SET field=? WHERE id=?</code>    |
| Delete        | <code>db.delete(obj); db.commit()</code>                    | <code>DELETE FROM table WHERE id=?</code>           |
| Count         | <code>db.query(Model).count()</code>                        | <code>SELECT COUNT(*) FROM table</code>             |

### FastAPI Cheat Sheet

| Feature       | Code                            |
|---------------|---------------------------------|
| Create app    | <code>app = FastAPI()</code>    |
| GET endpoint  | <code>@app.get('/path')</code>  |
| POST endpoint | <code>@app.post('/path')</code> |

|                  |                                                    |
|------------------|----------------------------------------------------|
| URL parameter    | @app.get('/items/{item_id}') def fn(item_id: int)  |
| Request body     | def fn(data: PydanticModel)                        |
| Database session | def fn(db: Session = Depends(get_db))              |
| Return 404       | raise HTTPException(status_code=404, detail='msg') |
| Run server       | uvicorn api:app --reload                           |

---

## Section 8 — What Comes Next

You have built a complete CRUD API with a real database. Here is the exact path forward to become job-ready.

### Immediate Next Steps

| Step | What                  | Time needed | Why it matters                                    |
|------|-----------------------|-------------|---------------------------------------------------|
| 1    | Switch to PostgreSQL  | 1 day       | What real companies use                           |
| 2    | Deploy on Railway     | 1 day       | Your API on a real URL — shareable                |
| 3    | JWT Authentication    | 1 week      | Secure your API — login, tokens, protected routes |
| 4    | AI Integration        | 2 weeks     | Add Claude/OpenAI to your endpoints               |
| 5    | Build 2 more projects | 1 month     | Portfolio — companies want to see work            |

### PostgreSQL Switch — One Line Change

```
# Step 1: Install PostgreSQL driver

pip install psycopg2-binary


# Step 2: Change ONE line in database.py

# Before:

DATABASE_URL = 'sqlite:///tasks.db'


# After:

DATABASE_URL = 'postgresql://username:password@localhost/taskdb'


# Everything else stays IDENTICAL

# Same engine, same SessionLocal, same models, same endpoints
```

### JWT Authentication — What to Add

```
# What JWT gives you:

POST /auth/register → create account

POST /auth/login → get a token
```

GET /tasks → only works with valid token in header

# Libraries you will use:

```
pip install python-jose passlib bcrypt
```

# Concept:

User logs in → server creates JWT token → user sends token with every request

→ server validates token → allows or denies access

## AI Endpoint — Your Differentiator

# What you will build:

POST /tasks/ai-prioritize

# Takes all your tasks

# Sends them to Claude/OpenAI API

# Returns prioritized list with reasoning

# Library:

```
pip install anthropic
```

# Basic pattern:

```
import anthropic
```

```
client = anthropic.Anthropic(api_key='your-key')
```

```
message = client.messages.create(
```

```
model='claude-sonnet-4-6',
```

```
max_tokens=1000,
```

```
messages=[{'role': 'user', 'content': f'Prioritize: {tasks}'}])
```

```
)
```

## Your Job-Ready Profile in 12 Months

|                                                      |
|------------------------------------------------------|
| Experience                                           |
| ■■■■■■■■■■                                           |
| Software Engineer — Defence Organization 2025 - 2027 |
| • Built real-time systems in C++, Qt, Sockets        |
| • Designed communication protocols                   |
|                                                      |
| Projects                                             |
| ■■■■■■■■■                                            |
| Task Manager API 2026                                |
| • FastAPI + SQLAlchemy + PostgreSQL                  |
| • Full CRUD, JWT auth, deployed on Railway           |
|                                                      |
| AI Task Prioritizer 2026                             |
| • FastAPI backend + Claude API integration           |
| • Autonomous task prioritization agent               |
|                                                      |
| Skills                                               |
| ■■■■■■■                                              |
| Python, FastAPI, SQLAlchemy, PostgreSQL              |
| Pydantic, JWT Authentication, REST APIs              |
| Claude/OpenAI APIs, LangGraph                        |
| C++, Qt, Sockets, Docker, Git                        |

Where to Apply

| Platform | Type           | Best for                                        |
|----------|----------------|-------------------------------------------------|
| Arc.dev  | Remote jobs    | Backend engineers — very Pakistan friendly      |
| Toptal   | Remote premium | Hard to get in — pays extremely well            |
| Upwork   | Freelance      | Start Month 6 — AI automation clients pay a lot |
| LinkedIn | Remote + local | Filter by Remote, Worldwide location            |

|               |                |                         |
|---------------|----------------|-------------------------|
| Rozee.pk      | Local Pakistan | Biggest local job board |
| Himalayas.app | Remote only    | Good AI engineer filter |

---

**You built a real backend. From scratch. While working full time.**

*PostgreSQL → Deploy → Auth → AI. That is your next move.*